

yarg-song-server

A self-hosted song library server for YARG (Yet Another Rhythm Game).

Point it at a folder of songs, run it on whatever is always-on in your house, and every YARG machine on the network plays from the same library. It is a single static binary — Docker, Raspberry Pi, macOS and Windows all run the same code.

> **Status: the server runs, and there are two ways to play from it.**
>
> It scans a library, serves a browsable and searchable catalog over HTTP, answers "which of these hashes am I missing?" in bulk, and hands out any song as a `.sng` — packing a loose folder on demand.
>
> - `yarg-sync` pulls a library into an ordinary songs folder that **unmodified YARG** then plays. No client changes, every platform YARG runs on. See `docs/SYNC-CLIENT.md`.
> - **The client fork** does the same thing from inside the game, on a folder it manages itself: point it at a server URL once and it mirrors on startup. See `docs/ADR-004-remote-song-source.md`.
>
> See `docs/ROADMAP.md` and `docs/API.md`.
>
> What "works" means here is measured, not asserted. Archives this tool writes are decoded by the reference `SngCli`; **a real YARG install scanned 22 archives served by the running server and accepted 20**, the two refusals being corpus cases built to be refused and both independently flagged by our own scanner. Details in `docs/TEST-CORPUS.md`.

Design in one paragraph

The server is a **content source, not a game modification**. It serves ordinary `.sng` files that an unmodified YARG already knows how to read, so the useful version needs no client changes at all — a small sync companion pulls the library into a normal songs folder. That remains the supported path, and it is the one that works on a YARG you did not build yourself.

The client fork adds the same thing natively, in the Unity layer, **without touching YARG.Core**, so it stays a candidate for upstreaming rather than a divergence.

Songs are identified by `SHA1(chart file bytes)`, which is exactly how YARG itself identifies them. Client and server therefore agree precisely on "do I already have this song", with no heuristics and no fuzzy matching.

What it will and will not do

Will

- Ingest loose song folders (`song.ini` + `notes.mid`/`notes.chart` + stems), `.sng` containers, and `.zip`/`.7z`/`.rar` archives of a song folder — a song packs to the same bytes whichever of those it arrived in, so reorganising a library does not make every client re-download it
- Serve a browsable, searchable catalog over HTTP, sorted by the same attributes YARG sorts by
- Serve songs as `.sng`, and answer "which of these hashes am I missing?" in bulk
- Run on `linux/amd64`, `linux/arm64` (Raspberry Pi), macOS and Windows — the arm64 half is measured rather than assumed: a Raspberry Pi 4 has run both the binary and the published container image, and the archives it served were byte-identical to the x86-64 ones (`docs/TEST-CORPUS.md`, "The ARM leg"), and **every commit to `main` executes the darwin build on a real Apple-silicon Mac** — server, sync client, and a rescan proving song identity survives the round trip — then checks that macOS and Linux agree hash for hash (`.github/workflows/macos-verify.yml`)

Will not, ever

- Decrypt Rock Band CON packages or encrypted moggs. Upstream lists CON decryption as out of scope and will reject PRs for it on sight; it also carries real legal exposure. `.con`, `_rb3con` and `.pkg` are refused on ingest with a clear message.
- Generate YARG's `songcache.bin`. Not because it is unreadable — it is a plain binary file — but because it stores **absolute local paths**, so a cache built here is meaningless on your machine; because its version is a date stamp checked with no compatibility window and no migration path, and its field layout is internal with no version of its own; and because getting any of it wrong fails *silently*, with YARG quietly rebuilding the cache while the tool appears to have worked.
- Distribute copyrighted audio. Charts and audio are separable by design.

Documentation

Document	What it covers
`docs/ROADMAP.md`	Phases, build order, exit criteria, blockers
`docs/API.md`	Every HTTP endpoint, what it promises, and what it explicitly does not claim
`docs/ADR-001-server-architecture.md`	Why Go, why two repos, why sync-first, why LGPL
`docs/ADR-002-v1-store.md`	Why the catalog lives in memory, why packed archives are cached, and the two places this server deliberately sorts differently from the client
`docs/ADR-003-archive-ingest.md`	Why `.zip`/`.7z` are ingested, what the `.7z` dependency actually costs measured rather than estimated, and why Rock Band packages are refused out loud

Document	What it covers
`docs/ADR-007-rar-ingest.md`	Why `.rar` is a full container rather than unpack-only, the measured access-order numbers that allowed it, why only the first volume of a set is an entry point, and why there is no committed fixture
`docs/ADR-004-remote-song-source.md`	How a YARG client could reach a remote library — the six things YARG.Core's code actually says about it, and why the first increment needs no change to YARG.Core at all
`docs/BUILD.md`	Building the client fork — what it costs, what "Success with three errors" meant, and the naming decision a beta has to settle first
`docs/SYNC-CLIENT.md`	`yarg-sync`: flags, what it refuses to touch, why files are named by chart hash, and the Windows Defender false positive
`docs/UPSTREAM.md`	What YARG's own contribution rules say, what is already in flight upstream, and the bugs and seams we have to offer them
`docs/DEPLOY-VAULT2.md`	Running the image as a container — the `chown 65532` and pool-path details that matter, registry auth, and the first end-to-end result off the dev machine
`docs/SOURCES.md`	What is already documented and where — **read this before reverse-engineering anything**
`docs/TEST-CORPUS.md`	Where test input comes from, and what a real YARG install said about it
`docs/research/yarg-song-formats.md`	The `.sng` binary layout, `song.ini` keys, the metadata model, song identity, and a difficulty assessment for every part of a Go reimplementation

Read the research document before writing any parser code. It is the reason the scope is what it is.

Getting it

Every pipeline builds a downloadable archive per platform — **linux** amd64/arm64/armv7, **macOS** amd64/arm64, and **Windows** amd64 — attached to the pipeline's `release` job together with `SHA256SUMS`. Each archive carries **both binaries** (`yarg-song-server` and `yarg-sync`), a README, the commented config template and the licence, inside a folder named after itself.

Unzip it and run `start-server.cmd` on Windows or `./start-server.sh` elsewhere: it creates `songs/` and `data/` beside itself, starts the server on `:8080`, and prints the address to open. Put songs in `songs/` and reload.

Artifacts from `main` are kept **30 days**; a **tagged** build keeps its archives permanently.

The build is reproducible, and that is measured rather than asserted. A Windows workstation and the Linux CI runner produced **byte-identical archives for all six platforms** from commit `b90cc3e` — same SHA-256, every file. So a checksum from `SHA256SUMS` means something a reader can actually check.

Four things have to hold for that, and each was a real defect first:

Same Go version	The pipeline pins it; a workstation on a different patch release produces different binaries.

`-trimpath`	Without it a binary embeds the builder's absolute paths — a plain `yarg-sync` carried **906 occurrences of the builder's home directory** — which also leaked a username into a publicly-mirrored artifact.
A genuinely clean tree	Go stamps `vcs.modified` into the binary. A tree that is dirty *only in git's opinion* still stamps `true`, and that flag alone accounted for 2,916 differing bytes.
LF everywhere (`.gitattributes`)	`web/browse.html` is `go:embed`ed, so a CRLF checkout changed **the page the server serves**: 15,653 bytes from a Windows build against 15,302 from Linux.

The archives themselves stamp a fixed timestamp rather than the clock, so repacking the same binaries always gives the same bytes.

The binaries are **not code-signed, and that is a decision rather than a to-do** — free software here does not carry a certificate subscription. Windows SmartScreen and macOS Gatekeeper will both object on first run, and that warning is accurate rather than spurious: nobody has paid a certificate authority to vouch for them. The archive's README says so and explains how to check the checksum. If you want a signed binary, the build is reproducible — build it yourself, sign it yourself, and you can prove it came from this source.

`docker pull registry.badassium.com/fatalexception/yarg-song-server:latest` is the other route, for amd64 and arm64. `:latest` **is the newest tagged release**, not the newest commit — so it is deliberately behind `main` most of the time, and it is a version that was chosen rather than one that merely compiled. `:main` is there if you actually want the newest development build, `:vX.Y.Z` names a release, and `:<eight-character sha>` names one exact build and never moves — **pin that one** for anything you intend to keep running.

Hosting it with Docker

`deploy/README.md` **is the one to read.** Two ways in:

You have	Use
Unraid	`deploy/unraid/my-yarg-song-server.xml` — a proper template, so the container gets an edit form, an update button and an autostart toggle instead of showing up as an orphan
Any other Docker host	`docker-compose.yml` — `SONGS_DIR=/srv/music docker compose up -d`

Two things apply either way, and the second is the one that bites silently:

- **The song library is mounted READ-ONLY.** The server never writes to it. It is somebody's music folder.
- `/data` **must be owned by** `65532:65532`**.** The image is distroless and runs as `nonroot`. Get it wrong and the server starts perfectly, serves the browse page, and then fails the *first time a song actually needs packing* — which reads as a broken song rather than a directory permission.

The image has **no shell in it**, so `docker exec` and Unraid's Console button cannot give you one. That is the distroless base doing its job, not a fault — use `/healthz` and the container log.

Read-only and unauthenticated by design: keep it on a LAN. Do not put it behind a reverse proxy and do not expose it to the internet.

Running it

```
make build
./bin/yarg-song-server --songs /path/to/songs --data ./data --listen :8080
```

`make package` builds every platform and produces the archives above into `release/`.

Then `GET /api/v1/library` to see what it indexed, `GET /api/v1/songs?q=&sort=artist` to browse, and `GET /song/{chart_hash}.sng` to pull a song. Every endpoint is in `docs/API.md`.

The library is read only ever; `--data` is where the server keeps its own state, including archives packed from loose folders, and must be writable.

It is unauthenticated and read-only. Run it on your LAN, not on the internet.

Features can also be turned on and off from the browse page itself, without a restart, when `config_writes` is set to `local` (this machine) or `lan` (anyone who can reach the port). **It is `off` by default** and it can never widen its own access. A change is written back to the config file the server read, so it survives a restart — one line, with every comment left intact. See `docs/API.md`.

One optional endpoint accepts an upload — `POST /api/v1/check` scans an archive and tells you whether it would work, keeping nothing. It is **off by default**; see `docs/ADR-005` for why that differs from the browse page, which is on.

Syncing a library into stock YARG

`yarg-sync` pulls a server's library into an ordinary songs folder. **YARG is not modified and does not know the server exists** — it sees files in a folder, which is all it has ever needed:

```
./bin/yarg-sync -server http://pi.local:8080 -songs "/path/to/YARG Songs" -dry-run
```

It writes `<chart_hash>.sng` files and nothing else, and never reads, renames or deletes anything it did not write — so it is safe to point at a folder that already holds your own charts. `-prune` is off by default. Every download is verified by re-deriving the song's identity from the bytes that arrived, so a truncated or substituted file fails closed rather than landing.

Full behaviour, including the deterministic answer when two packages share a chart, is in `docs/SYNC-CLIENT.md`.

> **Windows:** an unsigned Go binary that downloads files can trip a machine-learning malware verdict. Defender did exactly that to `yarg-sync` on 2026-09-05 — `Trojan:Win32/Bearfoos.A!ml` — and then stopped on its own within the hour once the classifier was revised upstream. If you hit it, re-test before doing anything drastic, and please don't add an antivirus exclusion or turn protection off to run this. Signing the release binary is the real fix and is on the list. Details and the escalation path are in the document above.

Configuration

Every setting can be a flag or a line in a config file, with the same name for both. A flag beats the file, the file beats the defaults, and `./yarg-song-server.conf` is read automatically if it exists:

```
./bin/yarg-song-server --write-config > yarg-song-server.conf
```

An unknown setting in that file is an error rather than a warning. A typo that is silently ignored leaves you certain you changed something you did not.

One setting is worth knowing about before you deploy: `pack_cache_max`, default 2 GiB. A song stored as a loose folder is packed to `.sng` on first request and the archive is kept, and an archive is within about 2% of the size of the folder it came from — so an unbounded cache over a loose library eventually needs a second copy of that library on the data disk. That is fine on a server and fatal on a Raspberry Pi whose `--data` is the SD card. Hitting the bound costs a re-pack and never data: the archive is rebuilt byte-identically and its package hash, which comes from the content, does not change. (Byte-identically because the archive's obfuscation mask is *derived* from the package hash. It was drawn at random until 2026-09-06, which made this sentence false and cost two clients a shared library; `docs/TEST-CORPUS.md` has the run that found it.) Raise it if you have the disk; `0` means unbounded and the server says so loudly at start if you choose it.

Building

```
make build      # host binary into ./bin
make test       # unit tests, with the race detector
make release    # every promised platform into ./dist
make docker     # multi-arch image
```

CI runs the same checks on every push — `gofmt`, `go vet`, the suite with and without `-race`, all six release targets, and an assertion that the Dockerfile's Go is not older than `go.mod` requires. On the default branch it also builds and pushes a **multi-arch container image** for `linux/amd64` and `linux/arm64`. See `.gitlab-ci.yml`.

That image needs no emulation: Go cross-compiles, and the Dockerfile pins its build stage with `FROM --platform=$BUILDPLATFORM`, so the toolchain runs at the host's architecture and Go emits the arm64 binary. If you are ever tempted to install qemu or register `binfmt_misc` to build this project, something else has gone wrong.

Trying the scanner directly

The CLI is still there, and is how the parsers get pointed at a real collection. Point it at a song library and it prints the catalog as JSON, one song per record:

```
./bin/yarg-song-server scan /path/to/songs
```

It recognises loose song folders and `.sng` archives, walks nested directories, skips folders that hold no chart, and reports an unreadable song on stderr rather than abandoning the scan.

Repacking a loose folder into a `.sng` also works:

```
./bin/yarg-song-server pack /path/to/song-folder out.sng
```

The chart is copied byte for byte, so the song's identity is unchanged. Archives written this way have been decoded by the reference `SngCli` and scanned by a real YARG install — see `docs/TEST-CORPUS.md`.

Licence

LGPL-3.0-or-later, matching YARG and YARG.Core. See `LICENSE`.

This project is not affiliated with or endorsed by YARC. It stands with upstream against piracy: it ships no content, and it will not help you play content you do not own.